

# Java Polymorphism Complete Notes

## 1. What is Polymorphism?

Polymorphism is one of the main concepts of Object-Oriented Programming (OOP). The word polymorphism means "**many forms**".

In Java, polymorphism allows one method, object, or reference to behave differently in different situations.

**Exam Definition: Polymorphism is the ability of an object or method to take many forms.**

## 2. Why Use Polymorphism?

- Improves flexibility
- Reduces duplicate code
- Supports method overriding
- Supports method overloading
- Improves scalability

## 3. Real World Example

One person can behave differently in different situations:

- Student in school
- Employee at office
- Child at home

One person → many forms = Polymorphism

## 4. Types of Polymorphism

| Type         | Another Name         | Achieved By        |
|--------------|----------------------|--------------------|
| Compile-Time | Static Polymorphism  | Method Overloading |
| Run-Time     | Dynamic Polymorphism | Method Overriding  |

## 5. Compile-Time / Static Polymorphism

Static polymorphism happens when the compiler decides which method to call.

**Static polymorphism is achieved using Method Overloading.**

## 6. Method Overloading Example

```
class MathOperation {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
}
```

```
double add(double a, double b) {  
    return a + b;  
}  
  
int add(int a, int b, int c) {  
    return a + b + c;  
}  
}
```

## 7. Explanation of Overloading

- Same method name
- Different parameters
- Compiler decides which method to use
- Faster than runtime polymorphism

## 8. Run-Time / Dynamic Polymorphism

Runtime polymorphism happens during program execution.

**Runtime polymorphism is achieved using Method Overriding.**

## 9. Method Overriding Example

```
class Animal {  
  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}
```

```
}

class Dog extends Animal {

    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {

    public static void main(String[] args) {

        Animal a = new Dog();

        a.sound();
    }
}
```

## 10. Explanation of Overriding

- Child class changes parent method
- Uses inheritance
- Decision happens during runtime
- Supports dynamic behavior

## 11. How Runtime Polymorphism Works

Parent reference can hold child object.

```
Animal a = new Dog();
```

JVM checks the actual object type during runtime and calls the overridden method.

## 12. Static vs Dynamic Polymorphism

| Feature             | Static       | Dynamic         |
|---------------------|--------------|-----------------|
| Decision Time       | Compile Time | Run Time        |
| Technique           | Overloading  | Overriding      |
| Speed               | Faster       | Slightly Slower |
| Inheritance Needed? | No           | Yes             |

## 13. Real Industry Use Cases

- Banking systems
- Game development
- Android applications
- Payment gateways
- Enterprise software

## 14. Industry Example

```
class Payment {  
  
    void pay() {  
        System.out.println("Generic Payment");  
    }  
}
```

```
}

class BkashPayment extends Payment {

    void pay() {
        System.out.println("Paid using Bkash");
    }
}

class CardPayment extends Payment {

    void pay() {
        System.out.println("Paid using Card");
    }
}
```

## 15. Advantages of Polymorphism

- Flexible code
- Easy maintenance
- Code reusability
- Cleaner architecture
- Scalable applications

## 16. Disadvantages of Polymorphism

- Complex for beginners
- Debugging can be harder
- Runtime polymorphism slightly slower

## 17. Internal Working

- Static polymorphism resolved by compiler
- Dynamic polymorphism resolved by JVM
- Uses dynamic method dispatch internally
- Method overriding requires inheritance

## **18. Last Minute Exam Notes**

**Polymorphism means many forms.**

**Static polymorphism = Method Overloading**

**Dynamic polymorphism = Method Overriding**

**Overloading happens during compile time.**

**Overriding happens during runtime.**

**Runtime polymorphism requires inheritance.**